SAECOIN — TESTNET OPERATIONS

Document: Testnet_Operations.md
Project: SAECOIN
Version: 1.0
Status: Draft — Implementation Pending
Network: Private Testnet / Public Testnet
Consensus: Proof of Work (PoW)
Transaction Model: UTXO
Development Environment: Windows 11, 16 GB RAM

---

1. PURPOSE

This document defines the operational procedures for creating, running, testing, monitoring, and maintaining the SAECOIN test network.

The testnet will be used to verify:

- Node startup and synchronization.
- P2P communication.
- Block production.
- CPU and GPU mining integration.
- Transaction creation and validation.
- Wallet operations.
- Chain reorganization handling.
- Difficulty adjustment.
- Network stability.
- Security behavior.

Testnet coins have no intended monetary value.

---

2. TESTNET OBJECTIVES

The testnet must demonstrate that:

1. Multiple independent nodes can communicate.
2. Nodes can synchronize blockchain data.
3. Miners can produce valid blocks.
4. Invalid blocks are rejected.
5. Transactions are validated consistently.
6. UTXO balances remain correct.
7. Wallets can send and receive test coins.
8. Nodes can recover after temporary disconnection.
9. Network failures do not silently corrupt blockchain state.

10. Software upgrades do not unexpectedly split the network.

---

3. NETWORK ENVIRONMENTS

SAECOIN development should use separate network environments.

3.1 Local Development Network

Purpose:

- Developer testing.
- Automated integration tests.
- Debugging.
- Consensus testing.

Characteristics:

- Runs on one development computer.
- Uses local node instances.
- Can use a short block interval.
- May use easy mining difficulty.
- Can be reset frequently.

3.2 Private Testnet

Purpose:

- Testing communication between multiple computers.
- Testing mining behavior.
- Testing wallet connectivity.
- Testing network recovery.

Characteristics:

- Restricted participants.
- Separate network identifier.
- Separate genesis block.
- Test-only coins.
- No real-money expectations.

3.3 Public Testnet

Purpose:

- Allow external developers and miners to participate.
- Test real network conditions.

- Discover compatibility and security problems.
- Evaluate node distribution.

Characteristics:

- Public peer discovery.
- Independent node operators.
- Public documentation.
- Public testnet explorer, if available.
- Test-only funds.

A public testnet should launch only after the local and private testnets are stable.

---

4. NETWORK SEPARATION

Mainnet and testnet must be technically separated.

Each network must have its own:

- Network identifier.
- Genesis block.
- P2P configuration.
- Address prefix or network marker.
- Data directory.
- Peer discovery configuration.
- Wallet network selection.

Transactions from one network must not be accepted as valid transactions on another network.

The network identifier must be included in the appropriate transaction-signing domain or equivalent replay-protection mechanism.

---

5. TESTNET PARAMETERS

The following are initial testing suggestions, not finalized mainnet parameters.

Parameter| Initial Testnet Suggestion
Network name| SAECOIN Testnet
Network identifier| Unique testnet ID
Genesis block| Separate testnet genesis
Block interval| 30–60 seconds
Mining| CPU first, GPU later
Premine| No public monetary value

Coins| Test-only SAC
Difficulty| Testnet-specific algorithm settings
Peer connections| Configurable
Data directory| Separate from mainnet

The testnet may use different block timing and reward settings from mainnet.

These differences must be explicitly defined in network configuration and consensus rules.

---

6. NODE TYPES

6.1 Full Node

Responsibilities:

- Validate blocks.
- Validate transactions.
- Maintain UTXO state.
- Communicate with peers.
- Synchronize blockchain data.

6.2 Mining Node

Responsibilities:

- Obtain candidate block data.
- Perform proof-of-work calculations.
- Submit discovered blocks.
- Monitor mining status.

A mining node must validate its own candidate block before accepting it into the local chain.

6.3 Wallet Client

Responsibilities:

- Generate and manage wallet keys.
- Display balances.
- Create transactions.
- Sign transactions.
- Broadcast transactions through a trusted node or supported interface.

6.4 Explorer Service

Responsibilities:

- Display validated block information.
- Display transaction details.
- Show confirmation counts.
- Show network statistics.

The explorer is an informational service, not an independent consensus authority.

---

7. INITIAL TESTNET TOPOLOGY

The first multi-node test should use at least three node instances.

Example:

Node| Role| Location
Node A| Bootstrap / Full Node| Development PC
Node B| Full Node / Miner| Development PC or second PC
Node C| Full Node / Wallet Connection| Second PC

For a more realistic test:

- Run nodes on separate computers.
- Use different internet connections.
- Avoid relying on one bootstrap node.
- Test peer recovery after the bootstrap node goes offline.

Running three processes on one computer is useful for initial testing, but it does not prove real-world network resilience.

---

8. NODE STARTUP PROCEDURE

The eventual SAECOIN node software should provide documented commands for:

1. Selecting a network.
2. Selecting a data directory.
3. Configuring peer connections.
4. Starting the node.
5. Displaying synchronization status.
6. Displaying block height.
7. Displaying peer count.
8. Shutting down safely.

Illustrative command design:

"saecoin-node --network testnet"

"saecoin-node --network testnet --data-dir ./data/testnet"

These are proposed command examples. They are not existing executable commands.

Before starting a node, verify:

- Correct software version.
- Correct network selection.
- Correct genesis configuration.
- Available disk space.
- Valid configuration file.
- No conflicting process using the required ports.

---

9. NODE SYNCHRONIZATION TESTS

Test the following scenarios.

Test 1 — Fresh Node Synchronization

1. Start a new testnet node.
2. Connect it to existing peers.
3. Download available blocks.
4. Validate each block.
5. Confirm that the node reaches the current chain height.

Expected result:

The node synchronizes without accepting invalid blocks.

Test 2 — Temporary Disconnection

1. Start two connected nodes.
2. Disconnect one node.
3. Continue mining on the connected network.
4. Reconnect the disconnected node.
5. Observe synchronization.

Expected result:

The disconnected node catches up and validates the chain.

Test 3 — Restart Recovery

1. Stop a synchronized node.
2. Restart it.

3. Load the existing database.
4. Resume peer communication.

Expected result:

The node recovers without corrupting its blockchain state.

Test 4 — Database Recovery

1. Use a test database.
2. Simulate an unexpected shutdown.
3. Restart the node.
4. Verify database consistency.
5. If necessary, perform a controlled resynchronization.

Expected result:

The node either recovers safely or reports a clear failure without silently using invalid state.

---

10. MINING TESTS

Mining should initially be tested with CPU mining.

GPU mining should be integrated only after the mining algorithm and validation logic are stable.

10.1 CPU Mining

Test:

- Miner startup.
- Hashrate reporting.
- Block discovery.
- Block submission.
- Reward calculation.
- Miner shutdown.

Expected result:

A CPU miner can produce blocks accepted by independent validating nodes.

10.2 GPU Mining

Test after CPU mining works:

- GPU detection.

- Driver compatibility.
- Hashrate reporting.
- Correct work generation.
- Block submission.
- GPU failure recovery.

GPU mining must produce results that pass the same consensus validation as CPU mining.

10.3 Mining Compatibility

Test CPU and GPU miners against the same node implementation.

Expected result:

Both mining methods follow identical consensus rules.

10.4 Mining Failure Tests

Simulate:

- Miner disconnection.
- GPU driver failure.
- Invalid proof submission.
- Stale work submission.
- Sudden mining-power changes.

Expected result:

Invalid or stale work is handled safely and does not corrupt node state.

---

11. TRANSACTION TESTING

Test the complete transaction lifecycle.

11.1 Basic Transaction

1. Create a test wallet.
2. Obtain testnet coins.
3. Create a transaction.
4. Sign the transaction.
5. Broadcast it.
6. Observe transaction propagation.
7. Wait for block inclusion.

Expected result:

The receiving wallet displays the correct confirmed amount.

11.2 Invalid Transaction

Test:

- Invalid signature.
- Missing inputs.
- Already-spent inputs.
- Invalid amount.
- Invalid fee.
- Malformed serialization.

Expected result:

Invalid transactions are rejected.

11.3 Double-Spend Test

1. Create two conflicting transactions spending the same UTXO.
2. Broadcast both through different nodes.
3. Observe mempool and block behavior.

Expected result:

The consensus system never accepts both conflicting spends into the same valid chain state.

11.4 Transaction Reorganization Test

1. Create a test transaction.
2. Include it in a block.
3. Simulate a competing valid chain.
4. Trigger a controlled reorganization.
5. Observe transaction and wallet state.

Expected result:

Wallet balances and transaction confirmations reflect the active valid chain.

---

12. WALLET TESTING

Test:

- Wallet creation.
- Address generation.

- Receiving funds.
- Sending funds.
- Transaction signing.
- Wallet encryption.
- Password verification.
- Backup creation.
- Recovery on another device.
- Invalid password handling.
- Network mismatch handling.

A wallet backup must be tested by restoring it into a separate test environment.

Never use valuable real-world funds during wallet recovery testing.

---

13. NETWORK FAILURE TESTING

Simulate the following:

Scenario| Expected Behavior
One peer goes offline| Other peers remain operational
Bootstrap node goes offline| Existing nodes continue where possible
Network partition| Separate chains may form temporarily
Network reconnects| Nodes follow defined chain-selection rules
Malformed peer message| Message is rejected safely
Excessive connections| Connection limits are enforced
High transaction volume| Resource limits are respected
Disk nearly full| Node reports a clear operational warning

Network partition tests are especially important for a PoW blockchain.

---

14. CHAIN REORGANIZATION TESTING

The testnet must verify:

- Short reorganizations.
- Multiple competing blocks.
- Competing chains of different cumulative work.
- Transaction removal from the active chain.
- UTXO rollback.
- UTXO restoration.
- Wallet confirmation updates.
- Explorer confirmation updates.

The chain-selection rule must be based on the finalized consensus specification.

Do not assume that the chain with the most blocks is necessarily the correct chain if the protocol selects based on cumulative work.

---

15. MONITORING

Each testnet node should expose or log:

- Current block height.
- Current block hash.
- Peer count.
- Synchronization status.
- Mining status.
- Estimated hashrate.
- Current difficulty.
- Block interval.
- Invalid block count.
- Reorganization depth.
- Memory usage.
- CPU usage.
- Disk usage.
- Network traffic.

Monitoring must not reveal private keys, seed phrases, or sensitive wallet data.

---

16. TESTNET EXPLORER

If an explorer is available, it should display:

- Current block height.
- Recent blocks.
- Block hashes.
- Transaction identifiers.
- Transaction confirmations.
- Mining reward information.
- Network status.

Explorer information must be derived from validated node data.

The explorer must clearly identify the network as TESTNET.

---

17. TESTNET RESET POLICY

A private testnet may need to be reset during development.

A reset must:

1. Be explicitly authorized.
2. Identify the affected network.
3. Back up any data needed for debugging.
4. Remove only the intended testnet data.
5. Reinitialize the correct testnet genesis configuration.
6. Notify participating testers.

Never delete mainnet data during a testnet reset.

A public testnet reset must be announced clearly because it invalidates previous testnet balances and chain history.

---

18. TESTNET RELEASE MANAGEMENT

Every testnet software release should include:

- Version number.
- Release date.
- Supported network.
- Build instructions.
- Change summary.
- Known issues.
- Upgrade instructions.
- Cryptographic checksums.
- Release signature where supported.

Consensus-related changes require special attention.

If a change modifies consensus rules, all participating nodes must follow a documented upgrade or network migration procedure.

---

19. TESTNET SUCCESS CRITERIA

The testnet may be considered operationally successful when:

1. Multiple independent nodes communicate reliably.
2. Nodes synchronize from a fresh data directory.
3. CPU mining produces valid blocks.
4. GPU mining works if implemented.

5. Wallets send and receive test coins.
6. Invalid transactions are rejected.
7. Invalid blocks are rejected.
8. UTXO state remains consistent.
9. Controlled reorganizations work correctly.
10. Nodes recover after disconnection.
11. Database recovery tests pass.
12. No unresolved critical security issues remain.

Passing these tests does not automatically establish mainnet readiness.

---

20. TESTNET TO MAINNET TRANSITION

A successful testnet is only one requirement for mainnet.

Before mainnet:

- Finalize consensus rules.
- Verify monetary supply mathematics.
- Complete independent protocol review.
- Complete critical security testing.
- Freeze genesis parameters.
- Generate and independently verify the mainnet genesis block.
- Publish node software and checksums.
- Publish wallet software and recovery documentation.
- Establish release signing procedures.
- Publish network operating instructions.
- Establish incident response procedures.

Testnet data, balances, and keys must never be treated as mainnet assets.

---

21. FINAL PRINCIPLE

The testnet exists to discover problems before real users depend on SAECOIN.

A testnet that produces blocks is not necessarily secure.

A successful testnet must demonstrate correct validation, reliable synchronization, safe recovery, predictable mining behavior, and resistance to realistic failure scenarios.

No mainnet launch decision should be based only on successful block production.

END OF DOCUMENT